



Polymarket Perps

Security Review

Cantina Managed review by:

Giovanni Di Siena, Lead Security Researcher

Red Swan, Security Researcher

May 1, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
3.1	Medium Risk	4
3.1.1	Repeated balance roots inherit stale withdrawal accounting, blocking valid withdrawals	4
3.1.2	Contract-wallet accounts can be credited but have no viable withdrawal path	4
3.2	Low Risk	5
3.2.1	Chain-specific <code>block.number</code> semantics may cause <code>StateRootCommitted</code> to emit an incorrect block height	5
3.3	Gas Optimization	6
3.3.1	<code>processWithdrawal</code> uses memory-copying OpenZeppelin helpers instead of call-data variants	6
3.3.2	Unnecessary use of <code>nonReentrant</code>	6
3.4	Informational	7
3.4.1	Internal helper naming is inconsistent with the contract's underscore convention	7
3.4.2	Storage mappings omit semantic key/value names	7
3.4.3	Top-level NatSpec overstates transition-hash storage semantics	7
3.4.4	<code>withdrawn</code> NatSpec references an escape hatch that V1 does not expose	7
3.4.5	Superfluous remaining balance validation	8
3.4.6	Verbose validation logic in <code>setAsset()</code> can be simplified	8
3.4.7	<code>depositWithPermit()</code> silently discards <code>permit()</code> reverts	8
3.4.8	<code>_deposit()</code> emits a raw amount rather than a balance delta	9
3.4.9	Sequential state root batching with finalization delay to prevent immediate malicious commit usage	9
3.4.10	Operator trust assumptions around withdrawable balance and state root independent withdrawal execution	10

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

From Apr 25th to Apr 27th the Cantina team conducted a review of [Polymarket Perps](#) on commit hash [df69be71](#). The team identified a total of **15** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	2	2	0
Low Risk	1	1	0
Gas Optimizations	2	1	1
Informational	10	9	1
Total	15	13	2

2.1 Scope

The security review had the following components in scope for [Polymarket Perps](#) on commit hash [df69be71](#):

```
src/v1
├─ ExchangeV1.sol
├─ IExchangeV1.sol
└─ StorageV1.sol
```

3 Findings

3.1 Medium Risk

3.1.1 Repeated balance roots inherit stale withdrawal accounting, blocking valid withdrawals

Severity: Medium Risk

Context: (No context files were provided by the reviewer)

Description: ExchangeV1 keys withdrawal accounting by the Merkle root bytes:

```
mapping(bytes32 => mapping(address => mapping(address => uint256))) public withdrawn;
```

This implementation assumes that each root value uniquely identifies a withdrawal accounting period. However, the Merkle root commits only to the current state of account balances. Balance states can legitimately repeat—for example, if a user withdraws their balance and later deposits the same amount while no other balances in the system have changed.

When a previous root is re-committed, the contract reuses the stale `withdrawn[root][token][account]` values. If a user had previously exhausted their withdrawable limit under that root, they will be unable to withdraw their new, valid balance because the contract incorrectly calculates `alreadyWithdrawn` based on the previous epoch. This creates a liveness issue where valid withdrawals are blocked until the global state changes enough to produce a unique Merkle root.

Recommendation: Ensure withdrawal accounting is scoped to a unique commitment occurrence rather than the raw root bytes.

The recommended fix is to utilize the already existing `currentEpoch` in `StorageV1.sol` and increment it every time a new root is committed. The `withdrawn` mapping should then be updated to use this epoch as its primary key:

```
uint256 public currentEpoch;

mapping(uint256 Epoch => mapping(address Token => mapping(address Account => uint256
↪ AmountWithdrawn))) public withdrawn;
```

In `ExchangeV1.sol`, the `commitStateRoot` function should increment this epoch:

```
function commitStateRoot(bytes32 _stateRoot, bytes32 _hash) external virtual
↪ onlyOperator {
    if (_stateRoot == bytes32(0)) revert ZeroValue();

    stateRoot = _stateRoot;
    unchecked {
        ++currentEpoch;
    }

    emit StateRootCommitted(_stateRoot, _hash, block.number);
}
```

Finally, `processWithdrawal` and its internal settlement logic should be updated to use `currentEpoch` for all accounting lookups and updates. This ensures that even if a `stateRoot` repeats, the withdrawal accounting remains isolated to the current commitment period.

Polymarket: Fixed in commit [095a2d1](#).

Cantina Managed: Verified. Withdrawals now settle against the epoch which is incremented on every state root commitment.

3.1.2 Contract-wallet accounts can be credited but have no viable withdrawal path

Severity: Medium Risk

Context: (No context files were provided by the reviewer)

Description: The `ExchangeV1` protocol allows funds to be credited to arbitrary addresses, but its withdrawal mechanism is only compatible with Externally Owned Accounts (EOAs).

During the deposit process, the `_deposit` function allows a user to credit any to address without validation:

```
function _deposit(address token, uint256 amount, address to) internal {
    // ... validation ...
    emit Deposit(msg.sender, token, amount, to);
}
```

The offchain engine subsequently incorporates these addresses into the Merkle state root. However, the only way to withdraw these funds is through `processWithdrawal`, which relies on `_verifyWithdrawal` to validate the account owner's intent. This function requires a standard ECDSA signature:

```
address signer = ECDSA.recover(digest, signature);
if (signer != account) revert InvalidSignature();
```

Smart contract wallets-such as Gnosis Safes, multi-sigs, or EIP-4337 accounts-do not necessarily possess private keys and may require custom verification logic. Because `ExchangeV1` lacks support for signature validation standards like ERC-1271, any funds credited to a contract address are operationally stranded, as the account has no mechanism to authorize a withdrawal.

Recommendation: To resolve the mismatch between balance accounting and withdrawal capabilities, consider the following approaches:

1. **Support ERC-1271:** Implement EIP-1271 `isValidSignature` checks within `_verifyWithdrawal`. This is the most robust solution, as it enables compatibility with smart contract wallets by allowing them to authorize withdrawals via their own contract logic.
2. **Documentation and Best-Effort Validation:** Explicitly document that the V1 protocol is only intended for EOA use. While the contract could perform a best-effort check for `extcodesize == 0` during deposits to warn users, this is not a perfect enforcement mechanism because addresses can be empty at the time of deposit and later have code deployed (e.g., via `CREATE2`). Therefore, clear documentation and frontend-level warnings are the primary means of preventing users from accidentally stranding funds in contract wallets.

Polymarket: Fixed in commit [6410dd3](#).

Cantina Managed: Verified. ERC-1271 support has been added.

3.2 Low Risk

3.2.1 Chain-specific `block.number` semantics may cause `StateRootCommitted` to emit an incorrect block height

Severity: Low Risk

Context: [ExchangeV1.sol#L263](#)

Description: `commitStateRoot()` emits `block.number` in the `StateRootCommitted` event, implicitly assuming that this value represents the intended canonical block height for all deployments. However, this assumption is not portable across L2 networks due to inconsistent `block.number` semantics which could cause an incorrect value to be emitted.

Depending on the chain, it may refer to an L2 execution block, an approximate parent-chain block, or another chain-specific value. For example, [Arbitrum documents](#) that Solidity `block.number` returns a value close to the parent-chain block number, while the Arbitrum L2 block number is distinct and exposed through the `ArbSys` precompile.

Deployments to such L2s may emit a value that downstream consumers interpret incorrectly. This could cause systems relying on the event, such as off-chain indexers, monitoring systems, dispute/finality logic, etc, to reference the wrong chain state and associate a committed state root with an incorrect block height.

Recommendation: Carefully consider which chains this contract will be deployed to and avoid using `block.number` directly unless its semantics are correct for every supported deployment target. For multi-chain deployments, introduce a chain-aware `block.number` abstraction, such as a small wrapper

library, that returns the intended block number for each supported network. This adapter should use the appropriate chain-specific precompile or system contract where necessary, and deployments to unsupported chains should either revert or be explicitly configured.

Polymarket: Fixed in commit [208d719](#).

Cantina Managed: Verified. Use of `block.number` has been removed.

3.3 Gas Optimization

3.3.1 `processWithdrawal` uses memory-copying OpenZeppelin helpers instead of calldata variants

Severity: Gas Optimization

Context: *(No context files were provided by the reviewer)*

Description: The `processWithdrawal` function receives both `signature` (`bytes calldata`) and `proof` (`bytes32[] calldata`) as calldata parameters. However, the current implementation passes these directly to the standard OpenZeppelin `ECDSA.recover` and `MerkleProof.verify` helpers:

```
// ExchangeV1.sol:128
address signer = ECDSA.recover(digest, signature);

// ExchangeV1.sol:147
if (!MerkleProof.verify(proof, root, leaf)) revert InvalidProof();
```

Because these standard helpers are defined to take `bytes` memory and `bytes32[]` memory respectively, the Solidity compiler must copy the entire signature and Merkle proof from calldata into memory before the functions are executed. This results in avoidable gas overhead for every withdrawal processed onchain.

While this does not affect the security or correctness of the verification logic, it introduces a performance inefficiency in one of the most frequently called functions in the protocol.

Recommendation: To reduce gas costs, use the calldata-optimized variants provided by OpenZeppelin for these operations. These variants perform the verification directly against the calldata without requiring an intermediate memory copy:

1. **For ECDSA:** Replace `ECDSA.recover(bytes32, bytes)` with `ECDSA.recover(bytes32, bytes calldata)`. Note that in some versions of OpenZeppelin, this might be named `recoverCalldata`.
2. **For MerkleProof:** Replace `MerkleProof.verify(bytes32[], bytes32, bytes32)` with the equivalent that accepts calldata (e.g., `MerkleProof.verifyCalldata`).

Switching to these variants will optimize the gas efficiency of the withdrawal pipeline without changing the functional behavior of the contract.

Polymarket: Fixed in commit [468f80f](#).

Cantina Managed: Verified. The calldata variants are now used.

3.3.2 Unnecessary use of `nonReentrant`

Severity: Gas Optimization

Context: [ExchangeV1.sol#L280](#), [ExchangeV1.sol#L309](#), [ExchangeV1.sol#L329](#)

Description: The use of the `nonReentrant` modifier in `deposit()`, `depositWithPermit()`, and `processWithdrawal()` appears to be unnecessary due to strict conformity to the checks-effects-interactions pattern. All external calls occur after the necessary validation and state changes are performed, meaning there is no attack surface exposed for reentrant calls.

Recommendation: Consider removing the reentrancy guards.

Polymarket: Acknowledged. We prefer to retain the `nonReentrant` modifiers as a defensive measure.

Cantina Managed: Acknowledged.

3.4 Informational

3.4.1 Internal helper naming is inconsistent with the contract's underscore convention

Severity: Informational

Context: [ExchangeV1.sol#L134-L135](#)

Description: The `verifyProof` function is defined as `internal` but does not follow the leading-underscore naming convention used by all other internal helpers in the `ExchangeV1` contract.

Recommendation: Rename `verifyProof` to `_verifyProof` to match the established internal-helper style.

Polymarket: Fixed in commit [cc9cf5e](#).

Cantina Managed: Verified.

3.4.2 Storage mappings omit semantic key/value names

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: Several mappings in `StorageV1` are declared without naming their keys and values. This makes the storage model less self-describing, particularly for nested structures like `withdrawn`:

Recommendation: Use named mapping parameters to improve the readability and self-documentation of the storage layout. For example:

```
mapping(address operator => bool enabled) public operators;
mapping(address keeper => bool enabled) public keepers;
mapping(address token => uint256 minAmount) public supportedAssets;
mapping(bytes32 digest => bool used) public usedDigests;
mapping(bytes32 root => mapping(address token => mapping(address account => uint256
↪ amount))) public withdrawn;
```

Polymarket: Fixed in commit [76b82ce](#).

Cantina Managed: Verified. Named mapping parameters are now present.

3.4.3 Top-level NatSpec overstates transition-hash storage semantics

Severity: Informational

Context: [ExchangeV1.sol#L17-L21](#)

Description: The contract-level `@notice` for `ExchangeV1` states that the contract "stores hashes for full state transition data." However, the implementation only emits this hash via an event and does not persist it in storage.

Recommendation: Update the NatSpec to clarify that the transition-data hash is emitted via events rather than stored onchain.

Polymarket: Fixed in commit [3d47501](#).

Cantina Managed: Verified.

3.4.4 `withdrawn` NatSpec references an escape hatch that V1 does not expose

Severity: Informational

Context: [StorageV1.sol#L32](#)

Description: The NatSpec for the `withdrawn` mapping in `StorageV1.sol` describes it as tracking "ful-filled/processed/escape-hatch" amounts. However, no escape-hatch withdrawal mechanism exists in the V1 implementation; the only supported path is the keeper-processed flow.

Recommendation: Remove the "escape-hatch" reference from the NatSpec to accurately reflect the functionality available in V1.

Polymarket: Fixed in commit [aae0d00](#).

Cantina Managed: Verified.

3.4.5 Superfluous remaining balance validation

Severity: Informational

Context: [ExchangeV1.sol#L157-L158](#), [ExchangeV1.sol#L288-L289](#)

Description: `_remainingWithdrawable()` can revert for insufficient (negative/zero) balance but could instead return `(0, alreadyWithdrawn)` and revert at the call site in `processWithdrawal()`, which is already the case when `amount > remaining`. `SafeCast` is also not necessary given the balance sign is already checked.

Recommendation: Consider returning `(0, alreadyWithdrawn)` when the balance is negative/zero and remove the redundant `SafeCast`.

Polymarket: Fixed in commit [acede18](#).

Cantina Managed: Verified. `_remainingWithdrawable()` now returns the suggested tuple when the balance is negative/zero and the redundant safe cast has been removed.

3.4.6 Verbose validation logic in `setAsset()` can be simplified

Severity: Informational

Context: [ExchangeV1.sol#L243](#)

Description: The existing validation in `setAsset()` is unnecessarily complicated. It ensures that `_enabled` and `_minAmount` are consistent such that enabled assets must have a non-zero minimum amount, while disabled assets must have a zero minimum amount. The current condition expresses this relationship through two separate checks which can be simplified by directly comparing `_enabled` with whether `_minAmount` is non-zero.

This could be simplified further still by omitting the `_enabled` argument altogether. `_deposit()` simply checks whether a token has a non-zero minimum amount in `supportedAssets` storage to determine whether it should be considered enabled. As such, this can be specified without any need for the `_enabled` boolean, and the event can be inferred from `_minAmount`.

Recommendation: Consider simplifying the validation to `if (_enabled != (_minAmount != 0))`. Alternatively, `setAsset()` could be further simplified to:

```
function setAsset(address _token, uint256 _minAmount) external onlyOwner {
    if (_token == address(0)) revert ZeroAddress();

    supportedAssets[_token] = _minAmount;
    emit SupportedAssetModified(_token, _minAmount != 0, _minAmount);
}
```

Polymarket: Fixed in commit [5339cae](#).

Cantina Managed: Verified. An asset is now considered enabled when set with a non-zero minimum amount and disabled when it is zero.

3.4.7 `depositWithPermit()` silently discards `permit()` reverts

Severity: Informational

Context: [ExchangeV1.sol#L330](#)

Description: The current try/catch logic in `depositWithPermit()` does not distinguish between reasons that `permit()` could revert, for example on malformed or expired signatures. This could also occur for tokens that do not implement `permit`, with the notable exception of WETH which silently executes its fallback function without reverting. In all cases, the revert is silently discarded by the empty catch block. If the caller has not already approved sufficient balance then the subsequent transfer in `_deposit()` simply reverts without any indication as to why.

Recommendation: Consider explicitly surfacing the specific reason `permit()` calls fail by branching on the revert data bytes.

Polymarket: Fixed in commit [944e6c8](#).

Cantina Managed: Verified. The caller's allowance is checked before attempting a `permit()` call which now allows reverts to be surfaced.

3.4.8 `_deposit()` emits a raw amount rather than a balance delta

Severity: Informational

Context: [ExchangeV1.sol#L173](#)

Description: While it is explicitly documented that this contract should not be used with fee-on-transfer or rebasing tokens, it could still be preferable to enforce token invariants by emitting a balance delta rather than the raw specified amount.

Recommendation: Cache the contract balance of the specified token before the deposit and subtract this from the final balance to emit the true balance delta amount.

Polymarket: Fixed in commit [12a8c11](#).

Cantina Managed: Verified. The true balance delta is now emitted.

3.4.9 Sequential state root batching with finalization delay to prevent immediate malicious commit usage

Severity: Informational

Context: [ExchangeV1.sol#L261](#)

Description: While potentially partially included in the transition data, it could be prescient to consider storing committed batches, something like:

```
struct RootBatch {
    bytes32 root;
    bytes32 prevRoot;
    bytes32 dataHash;
    uint64 batchId;
    uint64 committedAt;
    uint64 finalizedAt;
}

uint64 public latestBatchId;
mapping(uint64 => RootBatch) public batches;
mapping(bytes32 => uint64) public rootToBatchId;
uint256 public finalizationDelay;
```

Then:

- Require monotonically increasing batches.
- Require reference to the previous root for all non-genesis batches.
- Add a finalization delay before a root can be used for withdrawals.

Conceptually:

```
function commitStateRoot(bytes32 newRoot, bytes32 prevRoot, bytes32 dataHash) external
→ onlyOperator {
    if (newRoot == bytes32(0)) revert ZeroValue();

    uint64 nextId = latestBatchId + 1;

    if (nextId > 1 && prevRoot != batches[latestBatchId].root) {
        revert InvalidPrevRoot();
    }
}
```

```

    batches[nextId] = RootBatch({
        root: newRoot,
        prevRoot: prevRoot,
        dataHash: dataHash,
        batchId: nextId,
        committedAt: uint64(block.timestamp),
        finalizedAt: uint64(block.timestamp) + finalizationDelay)
    });

    rootToBatchId[newRoot] = nextId;
    latestBatchId = nextId;

    emit StateRootCommitted(nextId, newRoot, prevRoot, dataHash);
}

```

This may help to mitigate the scenario in which both compromised operator and keeper can commit a bad root and have it used immediately.

Recommendation: Consider implementing the suggested batch relationships and a finalization delay.

Polymarket: Acknowledged. We consider the current version sufficient and may revisit the suggested enhancement in a future iteration.

Cantina Managed: Acknowledged.

3.4.10 Operator trust assumptions around withdrawable balance and state root independent withdrawal execution

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: The current withdrawal design relies on offchain operator accounting to determine whether a withdrawal is safe to approve and execute. In particular, the onchain state root only tracks user balances and does not independently differentiate margin, unrealized PnL, or withdrawable balance. Margin requirements and uPnL are instead accounted for by the operator, who is expected to approve withdrawals only against a user's withdrawable balance, calculated as the user's balance less margin and uPnL.

This creates an operator trust assumption: the correctness and safety of withdrawals depend on the operator accurately maintaining and enforcing offchain margin and PnL calculations at the time withdrawals are approved. If a user's PnL decreases, becomes negative, or is otherwise materially impacted between committed roots, for example due to funding fees or trading losses, a withdrawal that appeared safe at the time it was signed could later cause the account to become undercollateralized if not properly constrained by operator-side checks.

A related assumption exists because withdrawal signatures are not currently bound to a specific state root or batch. As a result, a signed withdrawal may be executed against whichever state root is current when the keeper submits the `processWithdrawal()` transaction. This means the withdrawal may execute under account conditions that differ from those present when the user signed the withdrawal, including conditions where the user's margin position has deteriorated. In such a case, the withdrawal could bring the account closer to undercollateralization or liquidation than the user may have intended when signing.

It is understood that the operator will mitigate this risk by checking withdrawals against the user's withdrawable balance, taking into account balance, margin, and uPnL. It is further understood that the operator will also check the user's available balance in the last committed state root to ensure the withdrawal does not violate the `remaining = provenBalance - alreadyWithdrawn` condition.

Recommendation: The protocol's safety depends not only on signature and Merkle proof validation, but also on the operator's correct enforcement of these offchain assumptions. Any failure, delay, or inconsistency in the operator's accounting of margin, uPnL, already-withdrawn amounts, or committed-root balances could allow withdrawals to execute under account conditions that are less collateralized than intended. This should be explicitly documented as an operator trust assumption that should be consistently and correctly enforced.

Polymarket: Fixed in commit [285724a](#).

Cantina Managed: Verified. Additional documentation on this trust assumption has been added to the operator guarantees section.